FRAMEWORK DATA SCIENCE ANTIGRAVITY

seagatejogja@ugm.ac.id

Standar MLOps Industri & Publikasi Akademik *Sistem eksekusi otomatis menggunakan Google Antigravity Agent terintegrasi dengan Systematic Literature Review (SLR) dan Protokol Anti-Leakage.*

BAB 1: PETUNJUK PENGGUNAAN (Standard Operating Procedure)

Framework ini mengubah IDE (seperti Google Antigravity) dari sekadar alat *coding* menjadi Asisten Riset dan *Machine Learning Engineer* otonom. Ikuti langkah berikut untuk setiap proyek baru.

Langkah 1: Persiapan Workspace (Satu Kali per Proyek)

1. Buka workspace Antigravity Anda.
2. Buka Terminal dan buat struktur folder standar dengan perintah: `mkdir -p data/raw data/processed src docs tests`
3. Pindahkan dataset mentah dari *stakeholder* ke dalam folder `data/raw/`.
4. Buat file `generate_skills.py` dan `prompt_generator.py` di direktori utama (salin kodenya dari Bab 2 dan Bab 3 di dokumen ini).
5. Jalankan perintah ini untuk membangun "Memori/Buku Pintar" bagi agen AI: `python generate_skills.py`

Langkah 2: Menyalakan "Control Center"

Alih-alih mengetik *prompt* manual, kita menggunakan antarmuka web khusus untuk merakit instruksi AI yang bebas kesalahan (*error-free*).

1. Di terminal, instal Streamlit (jika belum ada): `pip install streamlit`
2. Jalankan Control Center: `streamlit run prompt_generator.py`
3. Antarmuka web akan terbuka. Biarkan ini tetap menyala selama proyek berlangsung.

Langkah 3: Tarian 3 Langkah (Siklus Eksekusi)

Kerjakan proyek Anda secara bertahap dari Tab 1 hingga Tab 7 di antarmuka Streamlit. Untuk setiap fase, lakukan tarian berikut:

1. **ISI FORM:** Sesuaikan isian di antarmuka (misal: pilih Domain, masukkan rentang tahun SLR, dll).
2. **COPY PROMPT:** Blok teks *prompt* yang dihasilkan di kotak abu-abu pada bagian bawah layar, lalu *Copy*.
3. **PASTE KE AGENT:** Buka panel Chat Agent di Antigravity, *Paste* prompt tersebut, lalu tekan Enter. Biarkan agen menulis skrip, menjalankan terminal, dan membuat laporan `.md` secara mandiri. Lakukan tinjauan (*review*) pada laporan yang dihasilkan sebelum lanjut ke tab berikutnya.

BAB 2: SCRIPT PEMBANGUN "BUKU PINTAR" AGENT

Buat file bernama `generate_skills.py`, salin kode di bawah ini, lalu jalankan di terminal. Skrip ini akan membuat folder `.antigravity/skills/` yang berisi aturan mutlak (standar Q1 & Industri) agar agen AI tidak melakukan halusinasi atau memicu kebocoran data (*data leakage*).

Python

None
```
import os

def create_skill_files():
    skill_dir = ".antigravity/skills"
    os.makedirs(skill_dir, exist_ok=True)
    print(f"📁 Membangun direktori {skill_dir}...")

    skills = {
        "data-audit.md": """---
name: data-audit
description: Panduan wajib untuk EDA, audit data, dan deteksi
kebocoran (leakage).
---
# 📊 STANDAR AUDIT DATA & EDA (Q1 & INDUSTRY STANDARD)
```

```
1. **Rubin's Missing Data Framework:** Kategorikan missing value
menjadi MCAR, MAR, atau MNAR. Imputasi berbasis MNAR memerlukan
fitur indikator (is_missing).
2. **Leakage Taxonomy Audit:** - Hapus post-treatment features
(Target Leakage).
   - Pastikan ID Column tidak berkorelasi dengan target.
   - Deteksi Temporal Leakage (jangan gunakan random split pada
data waktu).
3. **Representativeness:** Cek survivorship bias atau class
imbalance.
""",

        "prep-pipeline.md": """---
name: prep-pipeline
description: Aturan besi Data Preparation, Sklearn Pipeline,
Multi-Stage FS, dan Imbalance.
---
# ⚙️ ATURAN BESI DATA PREPARATION (ANTI-LEAKAGE)
1. **SPLIT FIRST (Non-Negotiable):** `train_test_split` (atau
TimeSeriesSplit) SEBELUM fitting scaler/imputer apapun.
2. **Fit Only on Train:** Scaler/Imputer HANYA di-fit pada
`X_train`.
3. **Sklearn Pipeline Encaspulation:** Seluruh transformasi
terpusat pada Pipeline.
4. **Imbalanced Handling (SMOTE/Resampling):** WAJIB DI DALAM CV
fold (`imblearn.pipeline.Pipeline`).
5. **Multi-Stage Feature Selection:** Terapkan Filter
(VIF/Variance) -> Embedded (Lasso/Trees) -> Wrapper (RFECV) di
dalam struktur pipeline.
""",

        "modeling.md": """---
name: modeling
description: Eksperimen ML, Hyperparameter Tuning, dan Latency
Tracking.
---
```

```
# 🧠 STANDAR EKSPERIMEN MODELING & MLOPS
1. **SOTA Compliance:** Wajib mengikutsertakan algoritma/metode
yang direkomendasikan dalam dokumen SLR (Systematic Literature
Review) proyek ini.
2. **Baseline Wajib:** Mulai dengan `DummyClassifier/Regressor`.
3. **Tuning Protocol:** Gunakan Optuna/RandomizedSearchCV HANYA
pada `X_train`. DILARANG menggunakan Test Set untuk tuning!
4. **Industry Tracking (Wajib):** Log hasil wajib mencatat
'Inference Latency' (rata-rata waktu prediksi ms/baris).
""",

        "evaluation.md": """---
name: evaluation
description: Evaluasi Test Set, Confidence Intervals,
Calibration, dan Cost Matrix.
---
# 🏆 STANDAR EVALUASI FINAL (Q1 JOURNAL & ENTERPRISE)
1. **One-Time Use:** Test Set HANYA boleh diprediksi SATU KALI
oleh model pemenang.
2. **95% Confidence Intervals (CI):** WAJIB hitung CI menggunakan
Bootstrapping pada hasil `y_pred` vs `y_true`. (Jangan melatih
ulang model untuk bootstrapping).
3. **Probability Calibration:** Jika memungkinkan, evaluasi
Expected Calibration Error (ECE) atau Brier Score.
4. **Explainability:** Buat ringkasan SHAP (Global Summary &
Local Waterfall plot).
""",

        "reporting.md": """---
name: reporting
description: Penulisan Model Card, OOD Generalization, dan paper
akademik.
---
# 🎓 STANDAR PELAPORAN & DEPLOYMENT
```

```
1. **Google Model Card & Drift Monitoring:** Susun limitasi,
faktor evaluasi, dan rancangan pemantauan Data Drift
(PSI/KS-Test).
2. **Integritas Akademik (Q1 Reviewer Standard):**
   - Bahas potensi kelemahan lewat analisis "Out-of-Distribution
(OOD) Generalization".
   - DILARANG mengklaim kemampuan model melampaui rentang data
yang diobservasi.
   - Wajib mencantumkan bagian "Keterbatasan Metode/Dataset".
3. **Artifact Output:** Full pipeline diekspor rapi via
`joblib/ONNX`.
"""
    }

    for filename, content in skills.items():
        filepath = os.path.join(skill_dir, filename)
        with open(filepath, "w", encoding="utf-8") as f:
            f.write(content)
        print(f"✅ Berhasil membuat/memperbarui: {filepath}")

    print("\n🎉 SETUP SKILL FILES FINAL SELESAI!")

if __name__ == "__main__":
    create_skill_files()
```

BAB 3: SCRIPT CONTROL CENTER (PROMPT GENERATOR)

Buat file bernama `prompt_generator.py` dan salin kode ini ke dalamnya. Ini adalah aplikasi Streamlit yang akan menghasilkan *prompt* cerdas untuk Anda salin ke agen AI.

Python

None

```
import streamlit as st

# ==========================================
# KONFIGURASI HALAMAN
# ==========================================
st.set_page_config(page_title="Antigravity DS Prompt Generator
(Ultimate Edition)", layout="wide")

st.title("🚀 Antigravity Data Science Prompt Generator")
st.markdown("**Ultimate Edition:** Terintegrasi dengan
Multi-Domain Intelligence, SLR/SOTA Mapping, & Q1 Academic
Standards.")

tab1, tab2, tab3, tab4, tab5, tab6, tab7 = st.tabs([
    "Fase 2: Kickoff",
    "Fase 3: Data Audit",
    "Fase 3.5: SLR & SOTA",
    "Fase 4: Prep & Arsitektur",
    "Fase 5: Modeling",
    "Fase 6-7: Eval & XAI",
    "Fase 8: Publikasi"
])

# ==========================================
# TAB 1: FASE 2 - PROBLEM SCOPING
# ==========================================
with tab1:
    st.header("📝 Fase 2: Problem Scoping & Domain Intelligence")
    col1, col2 = st.columns(2)
    with col1:
        project_name = st.text_input("Nama Project",
value="Prediksi Risiko Berbasis Machine Learning")
        domain = st.selectbox("Domain Industri", ["Healthcare",
"Finance / Banking", "E-Commerce / Retail", "NLP / Text
Processing", "General / Lainnya"])
```

```
        task_type = st.selectbox("Tipe Task", ["Klasifikasi",
"Regresi", "Time-Series Forecasting", "Klasterisasi"])
    with col2:
        problem_desc = st.text_area("Deskripsi Masalah Bisnis",
value="Mengidentifikasi anomali atau risiko tinggi berdasarkan
data historis.")
        sensitive_data = st.radio("Apakah ada data sensitif
(PII/Medis)?", ["Ya", "Tidak"])

    st.divider()
    prompt_2 = f"""Kita mulai project data science baru.
- Nama Project: **{project_name}**
- Domain: **{domain}**
- Masalah: **{problem_desc}**
- Tipe Task: **{task_type}**
- Data Sensitif: **{sensitive_data}**

TUGAS ANDA:
1. Ajukan maksimal 5 pertanyaan elicitasi mengenai tujuan bisnis,
ketersediaan data, dan regulasi {domain}. Tunggu saya menjawab.
2. SETELAH saya menjawab, langsung buat dokumen CRISP-DM Project
Charter.
3. Tulis hasilnya ke file `docs/01_project_charter.md`. Jangan
cetak isi dokumennya di chat.
"""
    st.code(prompt_2, language="markdown")

# ==========================================
# TAB 2: FASE 3 - DATA UNDERSTANDING
# ==========================================
with tab2:
    st.header("📊 Fase 3: Data Audit & Leakage Detection")
    dataset_path = st.text_input("Path Dataset",
value="data/raw/dataset.csv")
    target_col = st.text_input("Nama Kolom Target",
value="target")
```

```
    st.divider()
    prompt_3 = f"""Fase 3 dimulai. Dataset: `{dataset_path}`.
Target: **{target_col}**.

TUGAS ANDA:
1. Baca `@data-audit.md`.
2. Buat script `src/eda_audit.py` untuk melakukan:
   - Klasifikasi missing values (MCAR/MAR/MNAR).
   - Deteksi class imbalance dan Audit Leakage Taxonomy.
3. Eksekusi `src/eda_audit.py` di terminal secara mandiri.
Lakukan self-correction jika error.
4. Tulis laporan komprehensif ke `docs/02_data_understanding.md`
lalu kabari saya di chat.
"""
    st.code(prompt_3, language="markdown")

# ==========================================
# TAB 3: FASE 3.5 - SLR & SOTA MAPPING
# ==========================================
with tab3:
    st.header("📚 Fase 3.5: Systematic Literature Review (SLR) &
SOTA Mapping")
    col_slr1, col_slr2 = st.columns(2)
    with col_slr1:
        search_keywords = st.text_input("Keywords Utama (Domain +
Task)", value="Machine learning classification SOTA")
        year_range = st.slider("Rentang Tahun Literatur", 2018,
2026, (2021, 2026))
    with col_slr2:
        slr_focus = st.multiselect("Fokus Pencarian SOTA",
["Feature Selection", "Class Imbalance", "Algorithm
Architecture", "XAI"], default=["Class Imbalance", "Algorithm
Architecture"])
    st.divider()
```

```
    prompt_3_5 = f"""Fase 3.5 dimulai. Kita wajib melakukan
Systematic Literature Review (SLR) mini.

Berdasarkan temuan di `docs/02_data_understanding.md`:
1. Lakukan sintesis literatur ilmiah untuk tahun
**{year_range[0]}-{year_range[1]}** dengan keywords:
**{search_keywords}**.
2. Fokuskan tinjauan Anda pada penemuan metode SOTA
(State-of-the-Art) untuk: **{", ".join(slr_focus)}**.
3. Evaluasi apakah ada arsitektur baru yang terbukti superior di
jurnal Q1 untuk kasus serupa dibandingkan algoritma klasik.
4. Susun matriks literatur dan rekomendasi final arsitektur
pipeline ke dalam dokumen `docs/03_sota_literature_review.md`.
5. Rekomendasi ini adalah landasan mutlak untuk penulisan kode di
Fase 4 dan 5!
"""
    st.code(prompt_3_5, language="markdown")

# ==========================================
# TAB 4: FASE 4 - PREPARATION & ARSITEKTUR
# ==========================================
with tab4:
    st.header("⚙️ Fase 4: Data Preparation (SOTA-Driven)")
    st.info("💡 Agen akan secara otomatis mengadopsi rekomendasi
preprocessing dari dokumen SLR.")
    st.divider()
    prompt_4 = f"""Fase 4 dimulai. Bangun arsitektur Pipeline
yang tangguh. Baca `@prep-pipeline.md` DAN rujukan SOTA di
`docs/03_sota_literature_review.md`.

TUGAS ANDA:
1. Buat modul Python `src/preprocessing.py`.
2. ATURAN MUTLAK: Buat fungsi `get_data_splits()` yang membagi
data (train/test) SEBELUM transformasi apapun.
3. Buat fungsi pabrik:
`create_experiment_pipeline(use_resampling=False, model=None)`.
```

```
   - Mengembalikan `imblearn.pipeline.Pipeline`.
   - ADOPSI METODE SOTA: Untuk strategi Imputasi, Feature
Selection, dan Resampling, Anda WAJIB menggunakan metode spesifik
yang direkomendasikan di dokumen SLR kita. Jangan gunakan default
jika literatur menyarankan metode lain.
4. Buat `tests/test_prep.py` untuk menguji leakage pipeline ini.
Eksekusi dan laporkan!
"""
    st.code(prompt_4, language="markdown")

# ==========================================
# TAB 5: FASE 5 - MODELING
# ==========================================
with tab5:
    st.header("🧠 Fase 5: Eksperimen & Latency Tracking")
    st.divider()
    prompt_5 = f"""Fase 5 dimulai. Baca `@modeling.md` dan
`docs/03_sota_literature_review.md`.

TUGAS ANDA:
1. Buat script `src/train_models.py` (import dari
`src/preprocessing.py`).
2. Latih `DummyClassifier/Regressor` sebagai baseline minimum.
3. Latih algoritma SOTA yang direkomendasikan di dokumen
literatur kita. Sertakan algoritma pembanding klasik.
4. Jika literatur menyarankan pendekatan spesifik untuk Imbalance
(Cost-Sensitive Learning vs Resampling), jadikan itu sebagai
skenario eksperimen perbandingan.
5. Tuning hiperparameter via Optuna dengan CV HANYA pada
`X_train`. Optimasi pada metrik yang tahan imbalance
(F1/MCC/PR-AUC).
6. **ATURAN MLOPS INDUSTRI:** Hitung 'Inference Latency'
(rata-rata waktu prediksi ms/baris).
7. Eksekusi script secara mandiri. Log metrik ke
`docs/experiment_register.csv`.
"""
```

```
    st.code(prompt_5, language="markdown")

# ==========================================
# TAB 6: FASE 6-7 - EVAL & XAI
# ==========================================
with tab6:
    st.header("🏆 Fase 6-7: Evaluasi Q1 & Ekspor Model")
    winning_model = st.text_input("Model Pemenang (Fase 5)",
value="Model SOTA Terpilih")
    st.divider()
    prompt_67 = f"""Fase 6-7 dimulai. Model pemenang:
**{winning_model}**. KITA BUKA TEST SET SEKARANG (HANYA SEKALI).

TUGAS ANDA:
1. Buat `src/evaluate_and_export.py`. Latih ulang model final
pada keseluruhan `X_train`. Prediksi `X_test`.
2. **Q1 Academic Standard:** Hitung metrik final, lakukan
Bootstrapping untuk **95% Confidence Interval (CI)** pada
prediksi, dan evaluasi Kalibrasi (ECE/Brier Score).
3. Analisis Explainability (SHAP).
4. Simpan model (joblib/ONNX) ke `data/processed/`. Eksekusi dan
tulis laporannya ke `docs/04_evaluation_and_model_card.md`.
"""
    st.code(prompt_67, language="markdown")

# ==========================================
# TAB 7: FASE 8 - PUBLIKASI
# ==========================================
with tab7:
    st.header("🎓 Fase 8: Laporan Akademik Q1")
    paper_title = st.text_input("Judul Paper", value="SOTA
Analysis for Predictive Modeling")
    st.divider()
    prompt_8 = f"""Fase 8 dimulai. Kumpulkan konteks proyek dari
folder `docs/`.
```

```
TUGAS ANDA:
1. Sintesis informasi menjadi paper akademik berjudul
**"{paper_title}"**.
2. Di bab **Tinjauan Pustaka**, integrasikan isi dari
`docs/03_sota_literature_review.md` untuk memperkuat justifikasi
pemilihan metode kita.
3. Di bab **Eksperimen & Hasil**, tampilkan CI 95% dan diskusikan
komparasi metode SOTA melawan baseline.
4. Wajib sertakan bab "Robustness & Out-of-Distribution (OOD)
Generalization Limits".
5. Tulis draf ke `docs/06_academic_report.md`. Kabari saya jika
selesai!
"""
    st.code(prompt_8, language="markdown")
```